

Document AI Fellowship Project

Complete Computer Vision Development Roadmap, Requirements, Architecture, Experiments, Evaluation and Deployment Plan

Project: Visual Document Understanding System

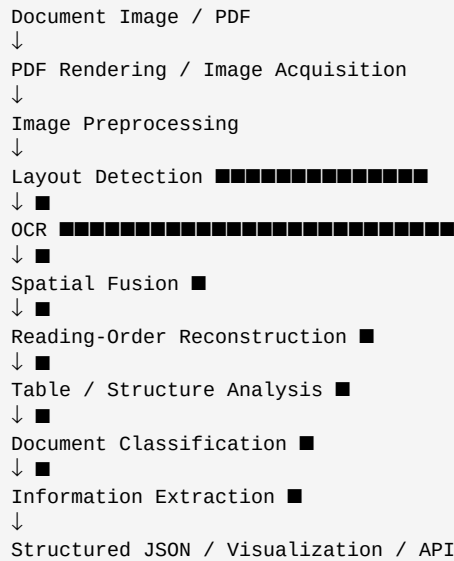
Primary objective: Build a production-oriented Document AI pipeline that combines document preprocessing, OCR, layout detection, spatial reasoning, reading-order reconstruction, table understanding, document classification, information extraction and deployment.

Recommended research direction: Study how document image quality affects visual document understanding and determine which preprocessing techniques improve OCR and layout detection performance.

Scope note: The project should be developed incrementally. The first milestone is a reliable OCR + layout-detection pipeline; advanced information extraction and deployment come afterward.

1. Project Definition

The proposed system accepts PDF files or document images and converts them into structured, machine-readable information while preserving the visual relationships between document regions and their text.



Primary project outcomes

- Detect document regions such as title, text, table, figure, header and footer.
- Extract text together with OCR bounding boxes and confidence scores.
- Associate OCR tokens with detected layout regions using spatial reasoning.
- Reconstruct a meaningful reading order, including multi-column layouts.
- Extract table content into structured rows and columns.
- Classify documents into categories such as resume, invoice, research paper, form or report.
- Extract selected semantic fields into JSON.
- Evaluate every major component independently and evaluate the end-to-end system.
- Deploy the system through a web interface and API.
- Conduct robustness experiments under blur, noise, low contrast, rotation and perspective distortion.

2. Why This Is a Strong Fellowship Project

- It develops practical computer vision fundamentals instead of relying only on image classification.
- It covers both classical computer vision and modern deep learning.
- It creates a clear research/experimental component.
- It provides measurable outputs: IoU, mAP, OCR error rates, extraction F1, latency and memory.
- It creates a strong visual demo through bounding boxes, OCR overlays and structured document views.
- It can later integrate with recruitment, legal-document, invoice-processing or enterprise-document systems.

What to avoid

- Do not make the project simply 'YOLO on documents'.
- Do not begin by connecting an LLM API and calling the result Document AI.
- Do not train everything from scratch without first establishing baselines.

- Do not randomly split pages from the same document source across train and test sets; this can create data leakage.
- Do not evaluate only final accuracy. Each pipeline component needs its own evaluation.

3. Prerequisites and Requirements

Programming

- Python; NumPy; Pandas; basic object-oriented programming; virtual environments; Git/GitHub; JSON; basic command-line usage.

Computer vision fundamentals

- Image representation, RGB/grayscale, image dimensions and coordinate systems.
- Resizing, normalization and interpolation.
- Gaussian and median filtering.
- Thresholding: binary, adaptive and Otsu.
- Morphological operations: erosion, dilation, opening and closing.
- Contours and connected components.
- Bounding boxes and coordinate transformations.
- Perspective transformation and homography.
- Image augmentation.
- IoU, NMS, precision, recall and mAP.

Deep learning

- CNN architecture and feature maps.
- Training/validation/test splits.
- Loss functions and optimizers.
- Learning rate and batch size.
- Overfitting and regularization.
- Transfer learning and fine-tuning.
- Classification and object detection concepts.

Document AI concepts

- OCR and OCR confidence.
- Document layout analysis.
- Reading-order reconstruction.
- Table structure understanding.
- Document classification.
- Named-field/information extraction.
- Spatial relationships between text and regions.

4. Hardware and Software Requirements

Component	Recommended preparation
GPU	6 GB GPU is sufficient to start; use small models/batches and reduce image size if necessary.
RAM	16 GB is comfortable; 8 GB can work with smaller datasets.
Storage	At least 20–30 GB free for datasets, model weights and experiments.
OS	Windows/Linux/macOS; Linux is preferred for reproducible ML environments.
Python	Use a dedicated virtual/conda environment.
Version control	Git + GitHub.
Annotation	CVAT or Label Studio.
Deployment	FastAPI + Streamlit + Docker.

5. Recommended Technology Stack

Layer	Tools
Language	Python
Deep learning	PyTorch, torchvision
Computer vision	OpenCV
Data	NumPy, Pandas
Visualization	Matplotlib
Object detection	Ultralytics YOLO
OCR	PaddleOCR and/or Tesseract baseline
PDF processing	PyMuPDF
Annotation	CVAT or Label Studio
Evaluation	scikit-learn plus custom IoU/OCR metrics
Experiment tracking	CSV/JSON initially; MLflow later if useful
Backend	FastAPI
Frontend	Streamlit
Deployment	Docker / Docker Compose

6. Complete Development Roadmap

Phase	Duration	Main output
Phase 0 — Environment and project setup	2–4 days	Create repository, environment, dependency file, project structure, Git workflow and baseline tests.
Phase 1 — Document preprocessing	5–7 days	Build grayscale, denoising, thresholding, CLAHE, deskew and perspective-correction pipeline.
Phase 2 — OCR baseline	5–7 days	Extract text, bounding boxes and confidence; create an OCR evaluation set.
Phase 3 — Dataset and annotation	5–10 days	Select diverse documents, define labels, annotate layouts and split by document/source.
Phase 4 — Layout detection	7–14 days	Fine-tune a pretrained detector and evaluate region detection.
Phase 5 — OCR + layout fusion	5–7 days	Associate OCR tokens with layout regions using spatial rules.
Phase 6 — Reading order	4–7 days	Implement reading-order reconstruction, including multi-column handling.

Phase	Duration	Main output
Phase 7 — Table understanding	7–10 days	Detect tables and convert table regions into structured rows/columns.
Phase 8 — Document classification	4–7 days	Train a transfer-learning classifier for document types.
Phase 9 — Information extraction	7–14 days	Extract selected semantic fields and produce structured JSON.
Phase 10 — Robustness research	7–10 days	Test blur, noise, contrast, rotation, perspective and compression effects.
Phase 11 — End-to-end evaluation	4–7 days	Evaluate each component and the complete pipeline.
Phase 12 — Deployment	7–10 days	Build Streamlit UI, FastAPI endpoint, Docker image and final demo.
Phase 13 — Documentation/presentation	4–7 days	Write report, experiment tables, architecture, limitations and resume-ready summary.

7. Phase 1 — Document Preprocessing

The preprocessing stage should improve document quality before OCR and layout analysis. Keep both the original and processed image so that later experiments can compare them.

```

Input PDF/image
→ render page
→ resize
→ grayscale
→ denoise
→ contrast enhancement (CLAHE)
→ thresholding
→ morphological cleanup
→ deskew
→ perspective correction
→ output clean image

```

Experiments

- Compare original versus grayscale.
- Compare Gaussian and median filtering.
- Compare global, adaptive and Otsu thresholding.
- Test CLAHE on low-contrast scans.
- Test morphological opening/closing for document noise.
- Estimate page rotation and deskew.
- Detect four page corners and perform perspective correction on photographed documents.

Deliverable

Notebook: 01_preprocessing.ipynb. It should display the original image and each preprocessing stage side-by-side or sequentially and record which transformation was applied.

8. Phase 2 — OCR Baseline

OCR should return more than plain text. Preserve the text, bounding box and confidence score for each token/line/word supported by the OCR engine.

```

OCR result:
[
{
"text": "John Doe",
"bbox": [120, 80, 420, 120],
"confidence": 0.97
}
]

```

Baseline comparisons

- Run OCR on original images.
- Run OCR on preprocessed images.
- Compare OCR output quality.
- Record character error rate (CER) and word error rate (WER) where ground-truth text is available.
- Record inference time.

Deliverable: 02_ocr_baseline.ipynb plus a small manually verified OCR benchmark.

9. Phase 3 — Dataset and Annotation

Recommended document diversity

- Research papers
- resumes
- invoices
- forms
- reports
- letters
- newspapers
- scanned documents
- documents containing tables
- documents containing figures

Suggested layout classes

TITLE
TEXT
TABLE
FIGURE
LIST
HEADER
FOOTER

Start with a manageable label set. Add classes only when the dataset contains enough examples to train them reliably.

Dataset split

Use approximately 70% training, 15% validation and 15% test, but split at the document/source level whenever possible. If pages from one PDF appear in both train and test, performance may be artificially inflated.

Annotation

Use CVAT or Label Studio. Annotate bounding boxes for the defined layout classes. Export into the format expected by the detector or write a conversion script.

10. Phase 4 — Layout Detection

Start from a pretrained YOLO model rather than training a detector from scratch. Fine-tune on your document-layout dataset.

Concepts you must understand

- Bounding-box coordinates.
- Confidence score.
- IoU.
- NMS.
- Precision and recall.
- AP and mAP@0.5.
- mAP@0.5:0.95.
- Training/validation curves.
- False positives and false negatives.

Training sequence

```

Pretrained detector
↓
Small sanity-check dataset
↓
Baseline fine-tuning
↓
Validation
↓
Error analysis
↓
Augmentation/hyperparameter experiments
↓
Final test evaluation

```

Experiments

- Image size
- batch size
- learning rate
- augmentation strength
- model size
- confidence threshold
- NMS/IoU threshold

11. Phase 5 — OCR + Layout Fusion

This is one of the most important custom computer-vision components. The detector gives region boxes and OCR gives text boxes. You must associate them.

```

Layout region:
TABLE = [100, 300, 700, 600]

OCR token:
"Price" = [400, 320, 500, 350]

→ determine whether the OCR box belongs to the TABLE region

```

Initial spatial strategies

- Center-point containment: assign a token to the region containing its center.
- Box overlap: assign using OCR/layout IoU.
- Distance to region boundary/center for ambiguous cases.

- Use confidence thresholds and keep unresolved tokens rather than forcing every token into a region.

12. Phase 6 — Reading-Order Reconstruction

A document is not simply a list of bounding boxes. The system needs a logical reading order.

Baseline algorithm

1. Group nearby text regions.
2. Sort primarily by vertical position.
3. Within the same row, sort by horizontal position.
4. Detect columns when horizontal gaps are large.
5. Read one column before moving to the next.
6. Preserve headings before their associated text where appropriate.

Later, compare your heuristic against a manually annotated reading-order benchmark.

13. Phase 7 — Table Understanding

Treat table extraction as a separate module.

```
Detected table
↓
Crop table region
↓
OCR
↓
Detect rows/columns or cell boundaries
↓
Assign OCR tokens to cells
↓
Construct structured table
↓
CSV / JSON
```

Example structured representation: columns = [Product, Quantity, Price]; rows = [[Laptop, 2, \$1000], [Mouse, 5, \$25]].

Useful CV concepts

- Line detection
- horizontal/vertical morphology
- contours
- cell segmentation
- spatial assignment
- OCR alignment

14. Phase 8 — Document Classification

Add a document-type classifier using transfer learning.

```
Document image
↓
CNN backbone
↓
feature extraction
↓
classification head
↓
Resume / Invoice / Paper / Form / Report / Other
```

- Start with a pretrained ResNet/EfficientNet-like backbone.
- Track accuracy and macro F1.

- Inspect the confusion matrix.
- Test whether preprocessing changes classification performance.

15. Phase 9 — Information Extraction

Move from visual understanding to semantic structure. Keep the first version deterministic and limited in scope.

Example: invoice

```
Invoice Number: INV-1029
Date: 2026-08-12
Total: $1,240

→
{
  "invoice_number": "INV-1029",
  "date": "2026-08-12",
  "total": 1240
}
```

Example: resume

```
{
  "name": "...",
  "email": "...",
  "skills": [],
  "experience": [],
  "education": []
}
```

An LLM can be introduced later as an optional semantic extraction layer. It should not replace your core vision pipeline.

16. Phase 10 — Robustness and Research Experiments

This is the part that can elevate the project from an implementation exercise into a fellowship-quality study.

Controlled degradation

- Gaussian blur
- motion blur
- salt-and-pepper noise
- low contrast
- rotation
- perspective distortion
- JPEG compression
- uneven illumination

Measure

- OCR CER/WER
- layout mAP
- document classification F1
- table extraction accuracy
- field extraction F1
- end-to-end latency

Compare baseline processing against your preprocessing pipeline. Record the magnitude of improvement or degradation for each condition.

17. Evaluation Plan

Component	Primary metrics
Document classification	Accuracy, precision, recall, macro F1, confusion matrix
Layout detection	IoU, precision, recall, mAP@0.5, mAP@0.5:0.95
OCR	CER, WER, confidence, latency
Reading order	Order accuracy / manually verified sequence accuracy
Table extraction	Cell accuracy, row/column accuracy, exact table match where practical
Information extraction	Precision, recall, F1, exact match
End-to-end system	Task success rate, latency, memory, failure rate

Error analysis

- Collect false positives and false negatives.
- Categorize errors: blur, unusual layout, small text, multi-column layout, tables, figures, skew, poor contrast.
- Show qualitative examples in the final report.
- Do not hide failures; explain them and propose improvements.

18. Phase 11 — Deployment Architecture



UI features

- Upload PDF/image.
- Show original page.
- Show preprocessed page.

- Show layout bounding boxes with labels.
- Show OCR text and confidence.
- Show structured extracted information.
- Show extracted tables.
- Allow downloading JSON/CSV output.
- Display inference time.

API endpoints

```
POST /predict
POST /ocr
POST /layout
POST /extract
GET /health
```

19. Recommended Repository Structure

```
document-ai/
├── data/
│   ├── raw/
│   ├── processed/
│   ├── annotations/
│   ├── splits/
│   └── notebooks/
│       ├── 01_preprocessing.ipynb
│       ├── 02_ocr_baseline.ipynb
│       ├── 03_layout_analysis.ipynb
│       ├── 04_detection_training.ipynb
│       ├── 05_ocr_layout_fusion.ipynb
│       └── 06_evaluation.ipynb
├── src/
│   ├── preprocessing/
│   ├── ocr/
│   ├── detection/
│   ├── layout/
│   ├── extraction/
│   ├── evaluation/
│   ├── utils/
│   ├── models/
│   ├── configs/
│   ├── app/
│   ├── api.py
│   ├── streamlit_app.py
│   ├── tests/
│   ├── requirements.txt
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── README.md
```

20. Engineering Requirements

- Use Git from the first day.
- Keep raw data separate from processed data.
- Never commit large datasets or model weights directly unless intentionally using Git LFS or external storage.
- Use configuration files for paths, thresholds, image size and model parameters.
- Write reusable Python modules instead of putting all logic in notebooks.
- Keep notebooks for experiments and visualization.
- Write unit tests for preprocessing, box conversion, IoU, spatial assignment and JSON output.
- Log experiment settings and metrics.

- Set random seeds where reproducibility matters.
- Save model checkpoints and evaluation results with clear version names.
- Document limitations and known failure cases.

21. Milestones and Acceptance Criteria

Milestone	Acceptance criterion
M1: Environment	Repository runs, dependencies install, sample pipeline executes.
M2: Preprocessing	At least five preprocessing operations implemented and visually compared.
M3: OCR	Text + bounding boxes + confidence extracted; benchmark created.
M4: Layout	Detector identifies defined layout classes with measurable mAP.
M5: Fusion	OCR tokens can be assigned to layout regions.
M6: Reading order	Correct order on representative single- and multi-column examples.
M7: Tables	At least a defined subset of tables converted to structured data.
M8: Classification	Document categories predicted with reported macro F1.
M9: Robustness	Controlled degradation experiments completed.
M10: Deployment	PDF/image upload produces visual and structured output.
M11: Final	End-to-end benchmark, report, demo and reproducible repository completed.

22. Practical 8–10 Week Schedule

Week	Focus	Deliverable
1	Environment, CV revision, preprocessing	Preprocessing notebook + baseline images
2	OCR	OCR module + benchmark
3	Dataset + annotation	Dataset specification + annotations
4	Layout detection	First trained detector
5	Detection improvement	Final detector + error analysis
6	OCR/layout fusion + reading order	Integrated layout-text representation
7	Tables + document classification	Table prototype + classifier
8	Robustness experiments	Experiment results + plots
9	Information extraction + API	Structured JSON + FastAPI
10	UI, Docker, final evaluation	Demo + report + final model

23. Start-Right-Now Checklist

- Create the GitHub repository.
- Create a dedicated Python/conda environment.
- Create the repository structure.
- Install PyTorch, OpenCV, NumPy, Pandas, Matplotlib, PyMuPDF and an OCR package.

- Create a small raw document sample of approximately 20–50 pages/images.
- Implement PDF page rendering.
- Implement grayscale conversion.
- Implement denoising.
- Implement Otsu and adaptive thresholding.
- Implement CLAHE.
- Implement basic deskewing.
- Save original and processed outputs.
- Create 01_preprocessing.ipynb.
- Record observations rather than merely displaying images.
- Create a project README containing objective, scope and planned architecture.

24. Detailed First 7 Days

Day	Tasks
Day 1	Repository, environment, dependency installation, project structure, README and Git workflow.
Day 2	PDF/image loading, page rendering, resizing, RGB/grayscale conversion.
Day 3	Gaussian/median filtering, thresholding, Otsu and adaptive thresholding.
Day 4	CLAHE, morphology, noise removal and comparison experiments.
Day 5	Deskewing and perspective correction.
Day 6	Build reusable preprocessing module; test on multiple document types.
Day 7	Write observations, compare methods, commit code and prepare OCR baseline.

25. Suggested Research Questions

- How much does document preprocessing improve OCR accuracy on degraded documents?
- Which preprocessing technique works best for low-contrast scans?
- How does document quality affect layout-detection mAP?
- Does preprocessing improve OCR and layout detection equally?
- Which augmentation strategies improve generalization to unseen document layouts?
- How does model size affect layout-detection accuracy versus inference latency?
- How well does spatial OCR-layout fusion work for single-column versus multi-column documents?
- Which document categories are most difficult to classify and why?

26. Recommended Ablation Studies

Ablation studies help demonstrate that individual components contribute measurable value.

Experiment	Compare
Preprocessing ablation	No preprocessing vs grayscale vs CLAHE vs full pipeline
Augmentation ablation	No augmentation vs selected augmentations

Experiment	Compare
Model-size ablation	Small vs medium detector
Resolution ablation	Lower vs higher input resolution
Fusion ablation	Simple OCR output vs OCR + spatial layout fusion
Robustness ablation	Baseline vs preprocessing under blur/noise/rotation

27. Final Report Structure

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Related Work
6. Dataset
7. Methodology
8. Preprocessing
9. OCR
10. Layout Detection
11. OCR/Layout Fusion
12. Reading Order
13. Table Understanding
14. Document Classification
15. Information Extraction
16. Experimental Setup
17. Evaluation Metrics
18. Results
19. Robustness Experiments
20. Error Analysis
21. Deployment
22. Limitations
23. Future Work
24. Conclusion
25. References

28. Resume Positioning

Do not describe the project as merely an OCR application. Present it as a visual document understanding system.

Suggested project title: Visual Document Understanding and Information Extraction System

- Developed an end-to-end Document AI pipeline combining OpenCV preprocessing, OCR, YOLO-based layout detection and spatial reasoning to convert unstructured document images into structured representations.
- Implemented OCR-layout fusion and reading-order reconstruction using document-region and text bounding boxes, with evaluation using detection and OCR metrics.
- Conducted robustness experiments under blur, noise, low contrast, rotation and perspective distortion and analyzed accuracy-versus-latency trade-offs.
- Deployed the pipeline using FastAPI, Streamlit and Docker for interactive document upload, visualization and structured JSON/table extraction.

Only include technologies and metrics that you actually implemented and measured.

29. Common Mistakes to Avoid

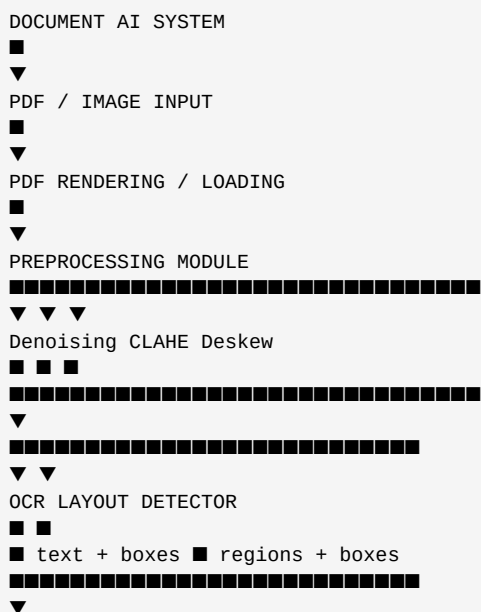
- Using a huge pretrained Document AI model without understanding the underlying task.
- Skipping a baseline before adding preprocessing or advanced models.
- Using only accuracy for imbalanced layout classes.
- Training and testing on pages from the same source document.

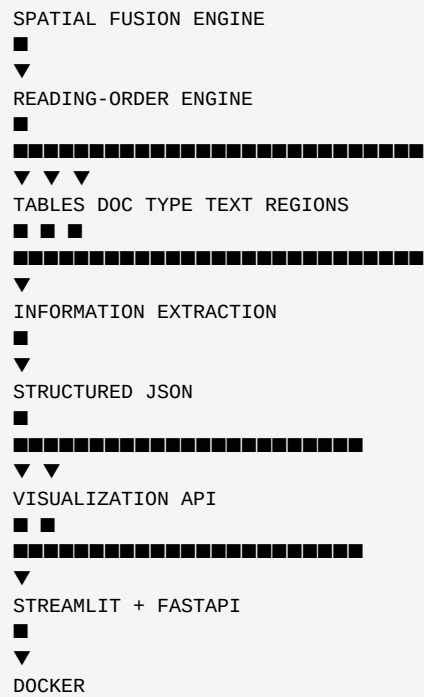
- Ignoring OCR bounding boxes and retaining only plain text.
- Not visualizing false positives and false negatives.
- Adding an LLM too early.
- Building the UI before the core pipeline is reliable.
- Changing multiple variables at once during experiments.
- Failing to record model versions, dataset versions and configuration settings.
- Reporting impressive metrics without explaining the test protocol.

30. Definition of Done

- The pipeline accepts PDF/image input.
- Preprocessing can be enabled/disabled for comparison.
- OCR produces text, bounding boxes and confidence.
- Layout detection produces labeled regions and confidence.
- OCR tokens are associated with layout regions.
- Reading order works on representative layouts.
- At least one table extraction workflow works end-to-end.
- Document classification has a measured benchmark.
- Robustness experiments are documented.
- The final system reports component-level and end-to-end metrics.
- The application can be run locally from documented instructions.
- Docker deployment works if Docker is included in the final scope.
- README explains installation, usage, architecture, datasets, models, metrics and limitations.
- Final report includes methodology, experiments, results and error analysis.
- The repository is reproducible enough for another person to run the main pipeline.

31. Final Architecture





Final guidance: Build vertically, not horizontally. Finish a small preprocessing → OCR → visualization pipeline first. Then add layout detection. Then fuse the outputs. Each phase should produce a working artifact, a benchmark and documented observations before the next phase begins.